

REPORT

Summary

This report is composed of an introduction session, description of the matrixmul, matmulperform programs and a summary of the observations made from the sequential and parallel executions graph.

Introduction

Matrix multiplication of say matrix1 and matrix2 can be achieved by multiplying each row of the first matrix by each column of the second matrix to obtain elements in the resultant matrix. Element indexed 'ij' (i for row and j for the column) of the resultant matrix is obtained by finding the summation of the multiplication of the ith row elements in matrix 1 by the jth column elements in matrix 2.

Some applications of matrices include representation of data in image processing, machine learning, etc.

The Matrixmul Program

- The program takes into consideration the matrix multiplication condition, that the number of columns in matrix1 should be equal to the number of rows in the matrix2. Matrices are represented using Python list data structure. Also, the program keeps matrix1 as the matrix with the largest number of rows for its row-based balancing algorithm.
- We devise an algorithm that allocates processes for parallel processing (subtle multiprocessing) when the number of requested processes (n) is greater than 1.
- The proposed loading-balancing algorithm in the form of per process row allocation works as follows: For a given n (number of processes requested),
 - It computes the number of rows to be allocated for each process as follows:
 - `alloc_per_process = int (len(matrix1) / num_process)` # row allocation per process;
where len(matrix1) is the number of rows in the largest matrix in terms of rows, num_process is the number of processes
 - The number of processes (num_process) can be computed as follows though supplied by the user:
 - `num_process = len(matrix1) // alloc_per_process` # number of processes
 - We also calculate the number of surplus processes that can be allocated to the last process (n-1) for n>2 ; to aid allocation of any extra process(es) that

may be left after equal distribution among requested processes. The last process may or may not have surplus processes allocated depending on the number of requested processes and the number of rows in the matrixes to be computed.

- `alloc_last_process_surplus = len(matrix1) - (alloc_per_process * num_process)` # row allocation for last process or first/last process if number of process is 1 or less than 2
- The proposed load balancing algorithm accepts a mode parameter that is set depending on the number of requested processes to compute matrixes either sequentially (when n is 1, mode is 'seq') or parallelly (when n is greater than 1, mode is 'ppl')
- In the parallel execution of matrix, the largest of the two matrixes is chosen as matrix1 while the other is matrix2.
- The matrixmul program has the following main functions embedded: `process_alloc()`, `matrix_mode()`, `matrix_compute()`, and the main Python functions.
 - The process alloc() function consists of:
 - A code section to spawn (fork) new processes from the main process by calling the Process class of the multiprocessing module. It passes as arguments the matrix_mode function, matrix1 and matrix2 as arguments in the form of tuples and a queue object to the Process class. Matrix1, in parallel execution, contains a batch of rows based on the number of rows per process allocation (alloc_per_process variable) that was passed in as an argument from the main function while matrix2 contains all elements read from the input file or randomly generated. For sequential execution, both matrix1 and matrix 2, contain full number of elements as supplied by the input set and a single process is created.
 - For parallel computation, it records the resultant matrix elements for the batch row jobs from the matrix_mode function after computation via the queue object method get(). For sequential mode, the results read from the queue contains all elements of the resultant matrix for all rows as contained in the batch jobs from matrix1.
 - The matrix_mode function operates by selecting the correct mode (either sequential – 'seq' or parallel – 'pll') and distributes the tasks based on row for matrix1 and complete set of elements for matrix2 to the matrix_compute function. For sequential mode, matrix1 is a complete set of elements in matrix1 while for parallel mode, matrix1 is a batch number of rows selected based on the alloc_per_process variable. In both modes, matrix2 always has the full set of elements.

- The `matrix_compute` function does a computation of `matrix1` (batch rows in `matrix1`) and `matrix2` by multiplying each *i*th row element by each *j*th column element for a resultant matrix element *ij*th. It does a similar computation for both sequential and parallel modes.
- The main function (`__main__`) performs the following duties to aid the matrix computation:
 - It reads the input file and extracts the matrix elements. After which it processes the matrix elements to remove, white spaces and other delimiters and conditions the matrix elements for computation.
 - It computes the rows to be allocated for each process (`alloc_per_process`), the number of processes and the number of surplus elements (`alloc_last_process_surplus`) that can be allocated to the last process in a scenario where we have the number of requested processes greater than 2 and available matrix rows cannot be evenly distributed among the processes. (e.g. for a 2 process allocation scenario with 11 rows to allocate, Process 1 gets 5 rows out of 11, with the remainder 6 rows allocated to Process 2).
 - It sets the type of start method needed to create or spawn or fork processes from the main process (`set_start_method('spawn')`). Also, it implements a unique buffer (FIFO) structure, that has in-built locks or semaphores to make consistency across processes and enable inter-process exchanges i.e. for process-safeness. Computed data results at each stage can of the matrix can be added to the queue or accessed from the queue (using `get()` and `put(item)` methods that can be called on a queue object).
 - The `process_alloc()` function is called and the various parameters `matrix1`, `matrix2`, queue object and computation mode (either `seq` or `pll`) is passed along with the function call.
 - Another utility function that is performed is its ability to print resultant matrix output to standard output and redirection of result to an output file (`Output.txt`)

The Matmulperform Function

- The `matmulperform` function is a variant of the `matrixmul` function with few differences listed below:
 - It generates a random $n \times n$ matrix using the `random` function where *n* is 50.
 - It encapsulates the details of the Python main (`__main__` function) , initially in the `matrixmul` function, in a user defined `main_function`.
 - It records the start and finish times at run time for both execution style modes (sequential or parallel) which is used to calculate the average CPU

runtime for the matrix computation task for $n = 1$ for sequential and $n = 2$ to $n = 50$ for parallel load distribution.

- Outputs of the average execution times (sequential and parallel) recorded at run time are saved in `para_exec.csv` and `seq_exe.csv` files to aid graph plotting on Excel.

Sequential versus Parallel Execution Plot

- The generated files '`seq_exe.csv` and `para_exec.csv`' were combined into file (`seq_para_exec.csv`) to aid plotting of the sequential versus parallel execution graph. Also, the combined file was processed to remove any white spaces, etc.
- The sequential execution plot is represented on the graph as a single point (blue marked) since the number of processes was one (1). Conversely, the parallel execution plot, which consists of processes from $n = 2$ to 5, is shown as a continuous line plot.
- The average execution time recorded for the sequential execution was approximately 0.222 seconds. A single process was spawned (forked) to handle the computation of all individual elements in the resultant matrix.
- Whereas the average execution time recorded for parallel execution showed values between the range of 0.220946789 – 0.268405676 seconds for process allocation of $n = 2$ to 13 depicting an undulating sequence due to the extra matrix row allocations to the last process (`alloc_last_process_supplus`). Between $n = 13$ to 35 processes allocation, the average execution time was constant for processes between 0.205182791 - 0.222909451 seconds. It showed the effect of increasing the number of processes on the average execution time, decreasing the average execution time approximately between 0.02 – 0.04 seconds. From $n = 35$ to 50, we further observed a slight undulating trend for the average execution time, depicting the effect of increased row allocation for the last process (for each n process). For the given matrix size 50×50 , processes in the range (around $n = 13$ – 35) had constant execution times, depicting an optimal process allocation range to handle the matrix multiplication computation task with good per process execution time for each n process.
- The average execution time for sequential execution was similar to the process allocation range ($n = 13$ to 17 processes) for parallel execution. With the parallel processing technique for matrix computation showing better overall execution time (between $n = 18$ to 31) as an optimal choice of number of processes to compute a given matrix size.

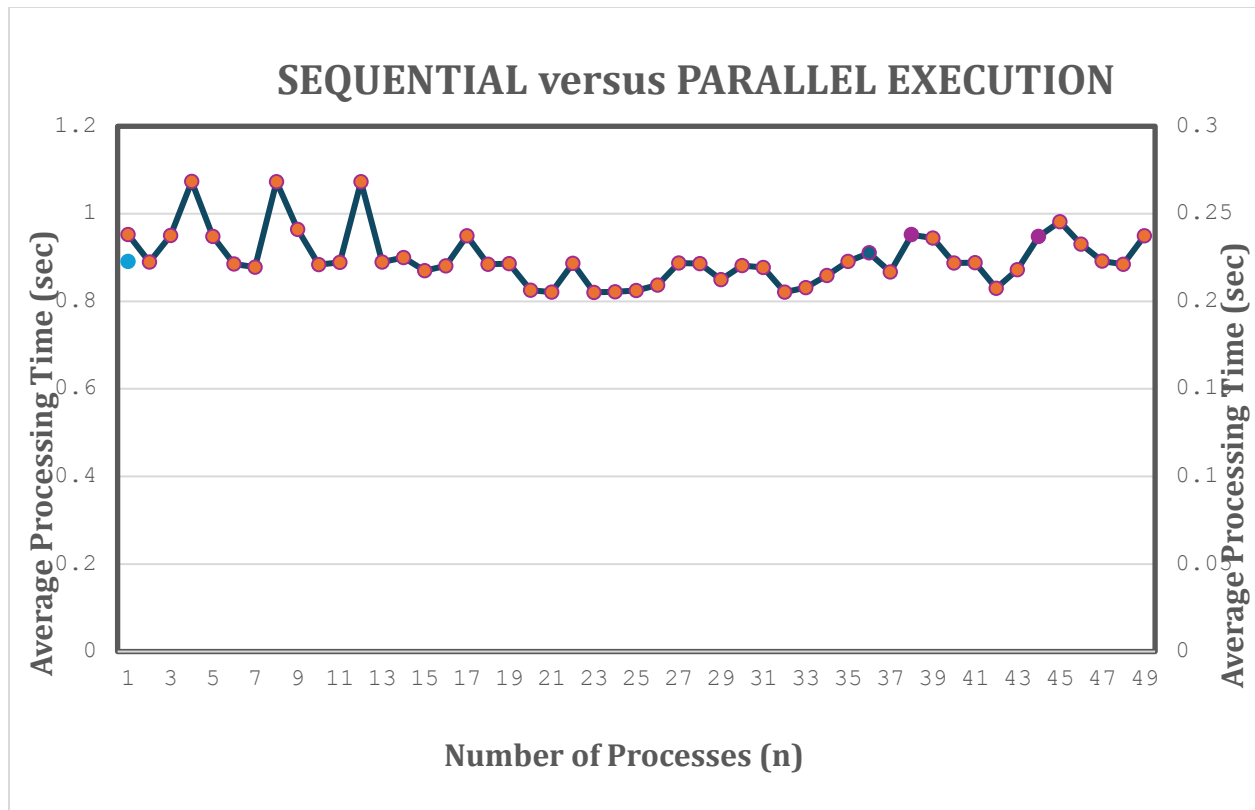


Fig. Sequential versus Parallel Execution line graph.

NB:

- The assignment was implemented using Python.
- Input file (input0.txt) and output file (output.txt) are in the same directory as python source file.
- Available CPU count is around ~8.
- Analysis of the graph is based on the secondary vertical axis (on the right side). The primary vertical axis (on the left side) is extrapolated from the secondary vertical axis to depict the observations since variations were in the order of 0.001 and less.